ENSEIRB-MATMECA

Mathematics and Mechanics Program

Study and implementation of h-adaptive mesh refinement for the simulation of one-dimensional multiphase flows.

Author:
KNEPFLER Yann

Internship Supervisor:
D. de Carvalho

Academic Advisor:
N. Barral

September 24, 2026



Abstract

This report first describes the equations and discretization methods used within the laboratory. Based on these, the relevance of the adaptive mesh refinement method for such problems is evaluated. Next, the adaptive mesh refinement approach is introduced and explained. The explanation covers the mesh structure used, how to handle this type of structure, and the procedure for modifying the mesh to achieve the desired refinement. To provide a clearer understanding of how these structures are manipulated, the implementation of the method is described, detailing the data structures used and key algorithms to clarify the role of the introduced parameter set. Finally, this method is tested on a simple test case to highlight the encountered challenges.

Contents

1 Physical Model **2**
- 1.1 Model Description . . . 2
- 1.2 Mathematical Formulation . . . 2

2 Numerical Method **3**
- 2.1 Discretization . . . 3
- 2.2 Test Case Example . . . 4

3 Adaptive Mesh Refinement Method **4**
- 3.1 Grid Configuration . . . 4
  - 3.1.1 Nested Grid Structure . . . 4
  - 3.1.2 Boundary Condition Definitions . . . 5
  - 3.1.3 Coarse Grid Corrections . . . 6
- 3.2 Mesh Refinement . . . 6
- 3.3 Error Estimator . . . 7

4 Implementation **7**
- 4.1 Data Structures . . . 7
- 4.2 Algorithms . . . 8

5 Application **9**
- 5.1 Configuration . . . 9
- 5.2 Validation . . . 10
- 5.3 Analysis . . . 10

6 Conclusion **11**

Introduction

To efficiently transport crude oil from subsea wells to offshore platforms, it is necessary to accurately determine key flow parameters of two-phase flows within the pipe network, such as pressure, phase velocities, and volume fractions. Given the large scale of pipe networks, the implemented numerical schemes are one-dimensional. The laboratory in which I conducted my internship seeks to implement high-order convergence methods to obtain more precise simulations. However, high-order schemes are not viable near solution discontinuities; thus, the order of accuracy is reduced near discontinuities. It therefore seems pertinent to refine the mesh around these discontinuities. Because these discontinuities propagate through the flow, using an adaptive mesh refinement method is particularly suitable.

1 Physical Model

1.1 Model Description

The model used is a four-equation model for compressible, isothermal two-phase flow in one dimension. In this model, the pressure in both fluids is assumed to be equal. This model is an adaptation of the one described in Wang, Gong, and Wu 2015.

1.2 Mathematical Formulation

The continuity equation for fluid k, where $k = 1, 2$, is given by:

$$\frac{\partial(\alpha_k \rho_k)}{\partial t} + \frac{\partial(\alpha_k \rho_k u_k)}{\partial x} = 0 \tag{1}$$

where α_k is the volume fraction of fluid k, ρ_k is the density of fluid k, and u_k is the velocity of fluid k along the pipe axis.

The momentum equation for fluid k is:

$$\begin{aligned}\frac{\partial(\alpha_k \rho_k u_k)}{\partial t} + \frac{\partial(\alpha_k \rho_k u_k^2)}{\partial x} = -\alpha_k \frac{\partial p_k}{\partial x} - (p_k - p_k^i)\frac{\partial \alpha_k}{\partial x} + \\ \alpha_k \rho_k \sin(\theta) + \frac{4\tau_{k,W}}{D_{H,k}} + \frac{\alpha \tau_i S_i}{A_k}\end{aligned} \tag{2}$$

where p_k is the pressure within fluid k, p_k^i is the interface pressure between the two fluids, θ is the pipe inclination, $\tau_{k,W}$ is the wall shear stress, $D_{H,k}$ is the hydraulic diameter, τ_i is the interfacial shear stress, S_i is the interfacial perimeter, and A_k is the cross-sectional area occupied by fluid k.

To close this system of equations, we have:

$$\alpha_1 + \alpha_2 = 1 \tag{3}$$

To ensure numerical stability, a pressure correction term Δp is introduced (Wang, Gong, and Wu 2015).

2 Numerical Method

2.1 Discretization

The discretization is performed on two staggered grids 1, where scalar variables are defined on the primary grid (solid line) and vector variables (velocities) are defined on the secondary grid (dashed line). This mesh configuration is described in Patankar 1980.

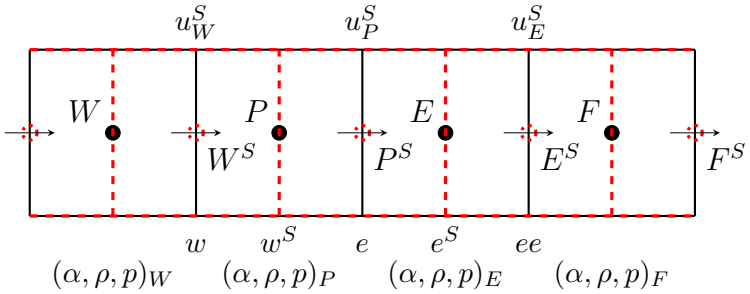


Figure 1: Staggered grids (adapted from Morales-Ruiz et al. 2012)

This grid configuration prevents odd-even decoupling phenomena that occur within pressure correction algorithms (Patankar 1980).

Prior to discretizing the equations, the advective terms in the momentum equations for each phase (2) are linearized to simplify numerical resolution (Wang, Gong, and Wu 2015), and the continuity equation multiplied by u_k is subtracted for both phases:

$$(\alpha_k\rho_k)\frac{\partial u_k}{\partial t} + (\alpha_k\rho_k u_k)\frac{\partial u_k}{\partial x} = -\alpha_k\frac{\partial p_k}{\partial x} - \Delta p\frac{\partial \alpha_k}{\partial x} + \alpha_k\rho_k g\sin\theta + \frac{4\tau_{k,W}}{D_{H,k}} + \frac{\alpha_k\tau_i S_i}{A_k} \quad (4)$$

After discretizing equation 4 on the secondary grid using the finite volume method, the left-hand side (LHS) is obtained as follows:

$$LHS = (\alpha_k\rho_k)_e^n\frac{u_{k,e}^{n+1} - u_{k,e}^n}{\Delta t} + (\alpha_k\rho_k u_k)_e^n\left(\frac{u_k}{\Delta x}\right)_e^n \quad (5)$$

Several numerical methods are implemented in the laboratory: two classic schemes—the first-order upwind scheme and the Monotone Upstream-centered Schemes for Conservation Laws (MUSCL scheme)—as well as a flux reconstruction scheme. The scalar variables α_k and ρ_k are evaluated on the secondary grid via interpolation.

The right-hand side (RHS) of equation (4) is discretized as follows:

$$\begin{aligned} RHS = -\alpha_{k,e}^n\frac{p_E^{n+1} - p_P^{n+1}}{\Delta x} - \Delta p_{k,e}^n\frac{\alpha_{k,E}^n - \alpha_{k,P}^n}{\Delta x} + \\ (\alpha_k\rho_k)_e^n g\sin\theta + \left(\frac{4\tau_{k,W}}{D_{H,k}}\right)_e^n + \left(\frac{\alpha_k\tau_i S_i}{A_k}\right)_e^n \end{aligned} \quad (6)$$

The continuity equations are discretized on the primary grid:

$$\frac{(\alpha_k\rho_k)_P^{n+1} - (\alpha_k\rho_k)_P^n}{\Delta t} + \left(\frac{\alpha_k\rho_k u_k}{\Delta x}\right)_P^n = 0 \quad (7)$$

2.2 Test Case Example

The implemented schemes are evaluated using the classic faucet problem (Harten 1983): in a vertical pipe filled with water and air, water is injected from the top. The water flow accelerates due to gravity and a pressure gradient, while air enters from the bottom of the pipe. Figure (2) shows that for this test case, a discontinuity propagates over time. Given that Godunov's barrier theorem guarantees that only a first-order scheme can preserve monotonicity in the presence of discontinuities, the implemented schemes locally reduce their order of accuracy near these discontinuities. Therefore, refining the mesh specifically around these discontinuous regions is essential to enhance the accuracy of the numerical schemes.

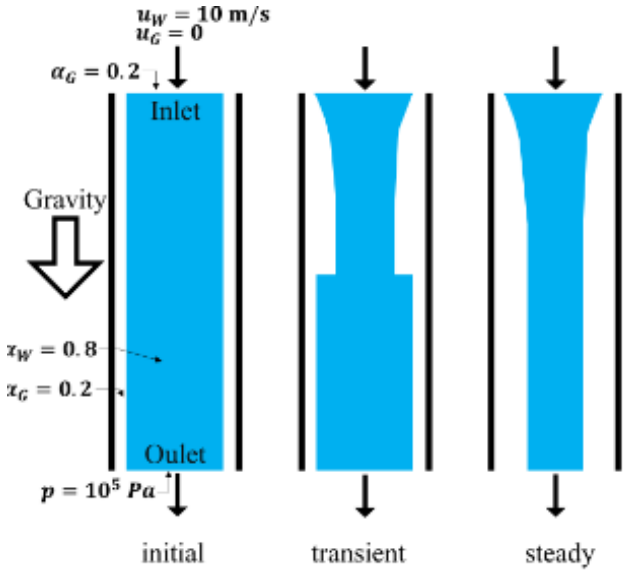


Figure 2: Diagram of the faucet problem (adapted from Morales-Ruiz et al. 2012)

3 Adaptive Mesh Refinement Method

The adaptive mesh refinement method consists of estimating regions generating the highest error—typically around steep gradients or discontinuities—and refining the grid locally in those regions. In general, this method is independent of both the specific system of equations and the chosen numerical scheme.

3.1 Grid Configuration

3.1.1 Nested Grid Structure

For adaptive mesh refinement, a hierarchy of nested uniform grids is used (M. J. Berger and Collela 1998). Refinement levels are denoted by $l = 1, ..., l_{max}$, where level 1 represents the coarsest grid, defined as:

$$G_l = \bigcup_k G_{l,k}. \tag{8}$$

A single grid exists at level $l = 1$, covering the entire domain. For each grid level, h_l denotes the cell width for grids at level l. A refinement ratio is defined as $r = \frac{h_{l-1}}{h_l}$, which must be an integer; here, $r = 2$ is selected.

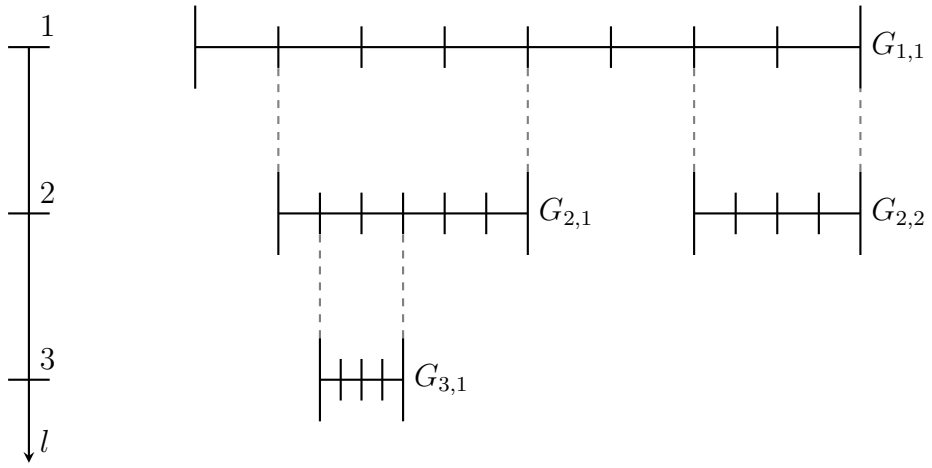


Figure 3: Nested grid structure

To ensure proper nesting among the grids, the following rules apply:

- A grid at level l must start and end at the corner of a cell belonging to the underlying level $l-1$ grid.
- A grid at level $l-1$ must separate a grid at level l from a grid at level $l-2$.
- A cell at level l can only neighbor cells belonging to levels l, $l-1$, or $l+1$.

We define the parent grid of $G_{l,k}$ as the level $l-1$ grid in which $G_{l,k}$ is "contained", and $G_{l,k}$ is called a child grid of $G_{l-1,q}$ if $G_{l-1,q}$ is its parent grid.

3.1.2 Boundary Condition Definitions

To advance a grid by one time step, boundary conditions must be specified. For this purpose, ghost cells are introduced at grid boundaries, as described in M. J. Berger and Collela 1998. The number of ghost cells required on each side depends on the stencil of the numerical scheme. Figure 4 highlights the ghost cells (dashed lines) of a refined grid $G_{l,k}$ for a problem defined on the domain $[0, 1]$. Let $U^n_{(m,q),i}$ denote the solution vector in cell i of grid $G_{m,q}$ at iteration n. Using Figure 4 as an example, ghost cells are assigned as follows:

- At the interface with the parent grid: $U^n_{(l,k),-1} = U^n_{(l,k),0} = U^n_{(l-1,q),I}$;
- At the physical domain boundary: values for $U_{(l,k),n+1}$ and $U_{(l,k),n+2}$ are determined using the physical boundary conditions.

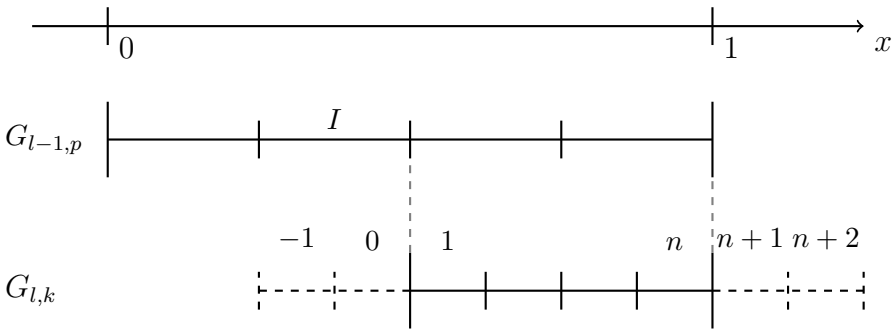


Figure 4: Grid representation with 2 ghost cells on each side

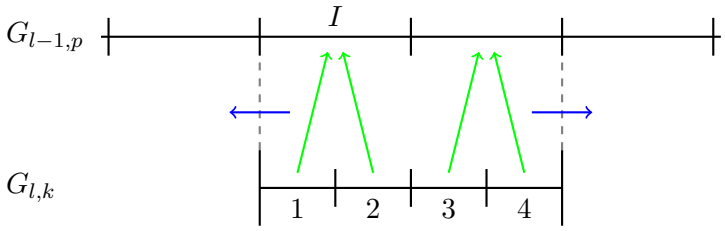


Figure 5: Correction of refined mesh cells

3.1.3 Coarse Grid Corrections

After integrating the governing equations on the child grids of a parent grid $G_{l,k}$, the refined regions of the parent grid are updated using the finer solution values. To maintain conservation, a flux correction is applied to the parent cells adjacent to the refined boundary (Marsha J. Berger and LeVeque 1998).

Taking Figure 5 as an example, the refined cells (green arrows) are updated according to:

$$U^{n+1}_{(l-1,p),I} = \frac{1}{2}(U^{n+1}_{(l,k),1} + U^{n+1}_{(l,k),2})$$

Let F^n_{i+1} denote the flux between cell m and cell $m+1$ on grid $G_{l,k}$ computed from the solution at iteration n. The fluxes for the coarse cells adjacent to the refined region (blue arrows) are corrected as follows:

$$U^{n+1}_{(l-1,p),I-1} := U^{n+1}_{(l-1,p),I-1} - \frac{k}{h_{l-1}}\left(F^n_{(l,k),1} - F^n_{(l-1,p),I}\right)$$

$$U^{n+1}_{(l-1,p),I+2} := U^{n+1}_{(l-1,p),I+2} + \frac{k}{h_{l-1}}\left(F^n_{(l,k),5} - F^n_{(l-1,p),I+2}\right)$$

3.2 Mesh Refinement

To refine a contiguous set of cells within grid $G_{l,k}$, a new child grid $G_{l+1,p}$ covering these cells is created. To initialize this child grid, values from the parent grid are prolonged such that:

$$\frac{1}{h_l}\int_{\text{parent cell}} U \mathrm{d}V = \frac{1}{h_{l+1}} \sum_{\text{child cells}} \int_{\text{child cell}} U \mathrm{d}V$$

(Marsha J Berger and Oliger 1984). Referring to Figure 6, prolongation is performed as follows:

$$U^n_{(l+1,p),0} = U^n_{(l,k),I} - \frac{1}{4}\text{slope}^n_I$$
$$U^n_{(l+1,p),1} = U^n_{(l,k),I} + \frac{1}{4}\text{slope}^n_I$$

where slope^n_I represents a slope limiter (here, the minmod limiter), defined by:

$$\text{slope}^n_I = \begin{cases} 0, \text{if } (\Delta U^n_{(l,k),I})(\Delta U_{(l,k),I+1}) \leq 0 \\ \text{sgn}(\Delta U^n_{(l,k),I}) \min(|\Delta U^n_{(l,k),I}|,\ |\Delta U_{(l,k),I+1}|), \text{otherwise} \end{cases}$$

and $\Delta U^n_{(l,k),I} = U^n_{(l,k),I} - U^n_{(l,k),I-1}$.

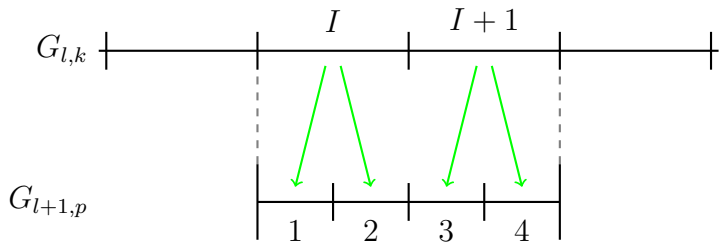


Figure 6: Prolongation onto a child grid

3.3 Error Estimator

An error estimator is employed to identify regions requiring refinement. In this work, an a posteriori error estimator is used, evaluating the error after integrating over the mesh. Specifically, Löhner's error estimator (Löhner 1987) is selected for its ability to detect discontinuities:

$$E_i = \frac{|U_{i+1} - 2U_i + U_{i-1}|}{|U_{i+1} - U_i| + |U_i - U_{i-1}| + \epsilon(|U_{i+1}| + 2|U_i| + |U_{i-1}|)}$$

where ϵ is a problem-dependent filter parameter designed to filter out noise and unphysical oscillations.

4 Implementation

The main objective was to develop a flexible library adaptable to various numerical schemes and error estimators. The resulting implementation is a Julia port of the Fortran90 code presented in Hawke 2018.

4.1 Data Structures

We define a grid structure containing solution data, references to parent and child grids, spatial coordinates, and its relative location within the parent grid. In addition, a level structure is defined, storing a list of all grids at the same refinement level, along with pointers to parent and child levels. Consequently, these structures form a tree hierarchy, as illustrated in Figure 7.

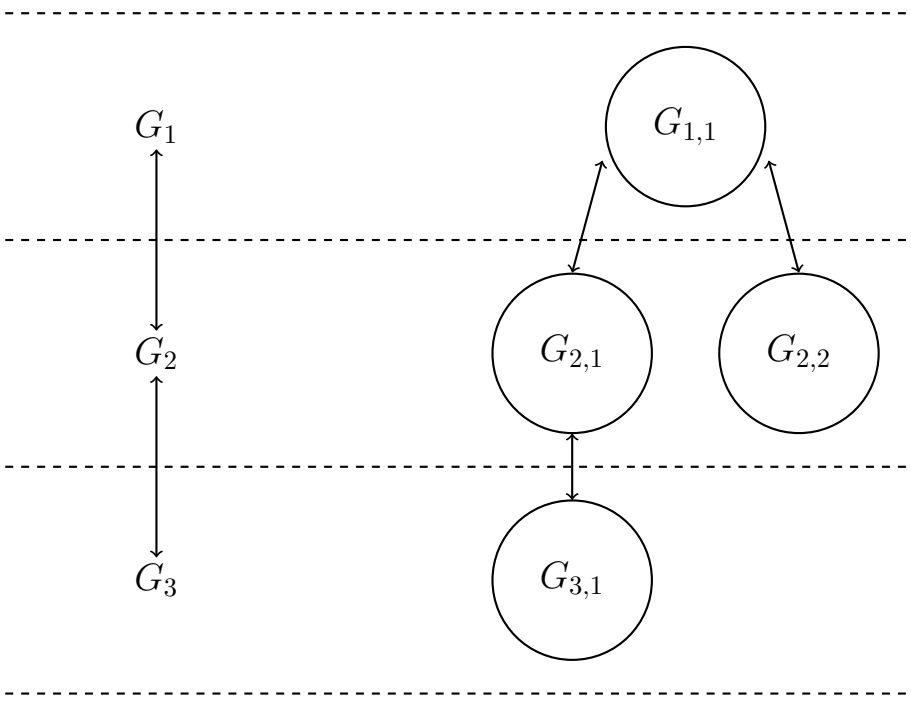


Figure 7: Data structures overview

4.2 Algorithms

To integrate a grid, its ghost cells must be fully initialized. Thus, level-wise time stepping proceeds from the finest level up to the coarsest level, as outlined in Algorithm 1. Because the pressure solver in equation (6) is implicit, ghost cell values at time level $n+1$ are required before advancing the equation at step n. To prevent unbounded recursion, a maximum refinement level l_{max} is enforced.

Algorithm 1 Level time integration

Données: niveau G_l
Intégrer la solution sur les maillages de G_l
if $l < l_{max}$ **then**
  Mettre à jour les mailles fantômes des maillages filles des maillages de G_l
  Appliquer cette algorithme au niveau G_{l+1}
  Corriger les mailles raffiner et les flux au bords des zones raffinées
  Applique l'algorithme de raffinement au niveau G_l
end if

It is advantageous to extend the refined zones slightly beyond strictly flagged cell boundaries to ensure that high-error regions remain within refined subdomains after time integration (Löhner 1987). This is achieved by introducing the `buffer_width` parameter in Algorithm 2. Furthermore, Algorithm 2 employs two distinct tolerances: a lower threshold for already-refined cells helps accommodate schemes prone to numerical diffusion. Indeed, numerical diffusion smoothes the solution, causing error estimates to decrease as integration progresses.

To prevent destroying a child grid that itself contains sub-grids, Algorithm 3 forces continued refinement in such cases by assigning an artificially elevated error estimate.

Algorithm 2 Level refinement

Data: niveau G_l, largeur de la zone tampon `buffer_width`, tolérance de raffinement `ref_tol`, tolérance de grossissement `coars_tol`
Appliquer l'estimateur d'erreur à l'ensemble des mailles de G_l
Maquer les mailles pour le raffinement à une distance de `buffer_width` des mailles dont l'estimation d'erreur dépasse `ref_tol` et des mailles précédement raffinées dont l'estimation d'erreur dépasse `coars_tol`.
Créer et prolonger de nouveaux maillages filles la où les mailles ont été marqués pour le raffinement
Copier les valeurs des anciens maillages filles sur les nouveaux
Associer les maillages du niveau G_{l+1} aux maillages G_{l+2}

Combined with buffer zones, this approach preserves proper grid nesting as defined in Section 3.1.1.

Algorithm 3 Error estimator algorithm

DATA: maille Ω
Appliquer la formule de l'estimateur de Löhner
Si Ω est raffinées au moins deux fois, mettre l'estimation d'erreur à 1

5 Application

5.1 Configuration

The implemented framework is evaluated on the 1D linear advective equation under several initial conditions:

$$\begin{cases} \frac{\partial u}{\partial t} + \frac{\partial u}{\partial x} = 0, \quad \text{where } x \in [0,1] \\ u(0,x) = u_0(x) \\ u(t,0) = u_0(-t) \\ u(t,1) = u_0(1-t) \end{cases} \tag{9}$$

The evaluated initial conditions are:

- Gaussian wave: $u_0(x) = 0.9 \exp\left(-\left(\frac{x-x_c}{\sigma}\right)^2\right) + 0.1$ where $x_c = 0.25$ and $\sigma = 0.05$;
- Step function: $u_0(x) = \begin{cases} 1, \text{if } x < 0.2 \\ 0, \text{otherwise} \end{cases}$;
- Square wave: $u_0(x) = \begin{cases} 1, \text{if } 0.1 < x < 0.2 \\ 0, \text{otherwise} \end{cases}$.

Time integration is performed using a first-order upwind scheme.

5.2 Validation

The framework is validated by measuring the convergence order for maximum refinement levels ranging from 1 to 4. The parameter set for this benchmark is:

- `ref_tol`: 0.00009375
- `coars_tol`: 0.00003125
- `buffer_width`: 4
- ϵ: 0.01
- CFL coefficient: 0.5

Convergence rates are evaluated solely on the Gaussian initial condition, as it is the only continuous function among the test set. Because the observed convergence order remains close to 1 across all four maximum refinement levels (Figure 8), the implementation is verified as valid. In Figure 8, the horizontal axis represents the cell size of the base (coarsest) grid.

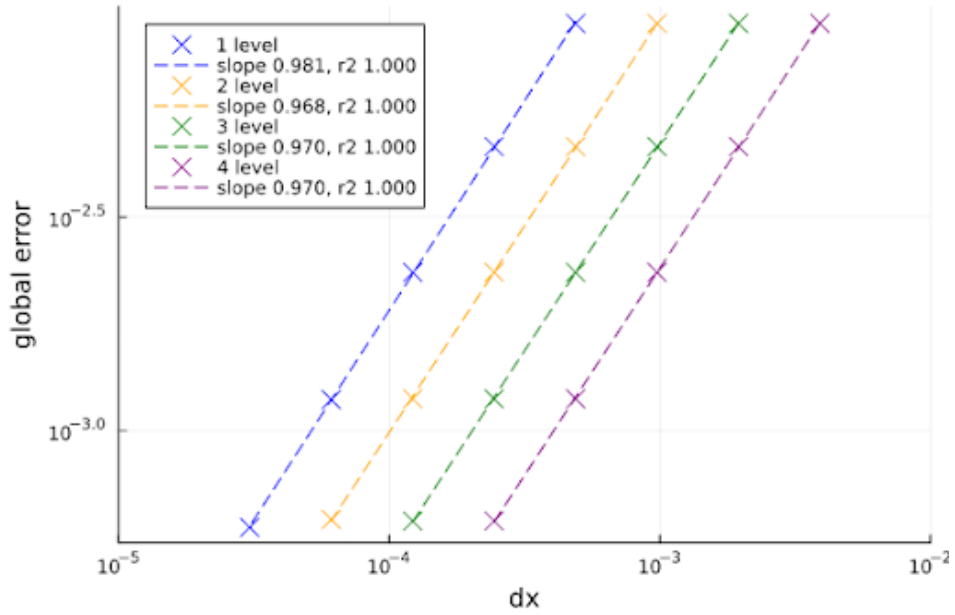


Figure 8: Convergence curve

5.3 Analysis

The framework is further evaluated across all three initial conditions by evaluating global error against different maximum refinement levels, l_{max}. Here, the number of base grid cells is set to 64, using the parameters from Löhner 1987:

- `ref_tol`: 0.3
- `coars_tol`: 0.1

- `buffer_width`: 4
- ϵ: 0.01
- CFL coefficient: 0.5

To ensure CFL stability across all refinement levels, the global time step is constrained by the cell size of the finest level. Since explicit Euler integration introduces significant numerical diffusion, increasing the maximum level amplifies diffusive errors on coarser levels. Consequently, Table 1 reveals minimal reduction in global error when increasing l_{max}. Furthermore, for this parameter set, the global error for the Gaussian profile is identical at $l_{max} = 3$ and $l_{max} = 4$. This is expected because at $l_{max} = 4$, only 3 refinement levels are actively triggered. This indicates that 3 refinement levels provide sufficient resolution so that estimated errors do not cross the refinement threshold. Hence, Löhner's error estimator more readily flags steep gradients associated with discontinuous solutions, fulfilling the objective stated in Section 2.2.

l_{max} / Initial Condition	Gaussian	Step	Square Wave
1	0.08012983661	0.09126679464	0.135470096
2	0.06978615294	0.07682885266	0.110086436
3	0.05254172054	0.0708716789	0.1009166435
4	0.05254172054	0.06958670085	0.09985460564

Table 1: Global error as a function of l_{max} and initial condition

6 Conclusion

Throughout this internship, the physical model and numerical schemes used by the laboratory were analyzed to understand the key challenges of adaptive mesh refinement for two-phase flows. The adaptive grid refinement technique was studied and subsequently evaluated on a 1D test case. At present, definitive conclusions regarding the efficiency of this adaptive mesh refinement approach when applied to the full laboratory problem cannot yet be drawn. However, selecting Löhner's error estimator appears to be a suitable choice, owing to its effectiveness in identifying discontinuities.

Perspectives for Future Work

To complete the evaluation of the implemented methodology, future work should test the framework using an implicit scheme and alternative error estimators to confirm modularity. To improve accuracy and limit excessive numerical diffusion, local subcycling in time should be implemented, allowing each refinement level to advance with its own time step while satisfying local CFL conditions.

References

Berger, M. J. and P. Collela (1998). "local Adaptive Mesh Refinement for Shock Hydrodynamics". In: *Journal of Computational Physics*.

Berger, Marsha J and Joseph Oliger (1984). "Adaptive mesh refinement for hyperbolic partial differential equations". In: *Journal of Computational Physics* 53.3, pp. 484–512.

Berger, Marsha J. and Randall J. LeVeque (1998). "Adaptive Mesh Refinement Using Wave-Propagation Algorithms for Hyperbolic Systems". In: *SIAM Journal on Numerical Analysis* 35.6, pp. 2298–2316.

Harten, Ami (1983). "High resolution schemes for hyperbolic conservation laws". In: *Journal of Computational Physics* 49.3, pp. 357–393.

Hawke, Ian (2018). *amr1d*. Southhampton, UK. URL: https://github.com/IanHawke/amr1d.

Löhner, Rainald (1987). "An adaptive finite element scheme for transient problems in CFD". In: *Computer Methods in Applied Mechanics and Engineering* 61.3, pp. 323–338.

Morales-Ruiz, S. et al. (2012). "Numerical resolution of the liquid-vapour two-phase flow by means of the two-fluid model and a pressure based method". In: *International Journal of Multiphase Flow* 43, pp. 118–130.

Patankar, S. V. (1980). *Numerical heat transfer and fluid flow.*

Wang, Z., J. Gong, and C. Wu (2015). "Numerical Simulation of One-Dimensional Two-Phase Flow Using a Pressure-Based Algorithm". In: *Numerical Heat Transfer; Part A: Application*. Wang et al (2015).